MASTER GUIDE

Java Computer Networks Lab

TCP & UDP - Client/Server Structures, All Programs & Algorithms

---

Repository: u2403262-Tejassh / Java_CN_Lab_TCP_UDP_All

36 Java programs - 9 lab experiments - each in TCP and UDP versions

Date: 30 September 2026

Contents of this guide

Part 1 - TCP and UDP Foundations: the common server & client structure used by every program

Part 2 - All 9 programs: explanation, how they work, TCP vs UDP differences and complete step-by-step algorithms (Step 1: Start ... Step n: Stop)

Part 3 - Quick reference: port table, ARQ comparison, compile & run instructions

Programs covered: Calculator, Chat, DVR (Distance Vector Routing), Stop-and-Wait ARQ, Go-Back-N ARQ, Selective Repeat ARQ, FTP, SMTP, Leaky Bucket.

Table of Contents

PART 1 - TCP AND UDP FOUNDATIONS 4

1.1 TCP vs UDP at a glance 4

1.2 Common TCP SERVER structure 4

1.3 Common TCP CLIENT structure 5

1.4 Common UDP SERVER structure 5

1.5 Common UDP CLIENT structure 5

1.6 Common mechanisms used across the programs 6

PART 2 - THE 9 PROGRAMS AND THEIR ALGORITHMS 7

2.0 Repository map - files and ports 7

2.1 Calculator (Calculator_TCP / Calculator_UDP) 7

2.2 Chat (Chat_TCP / Chat_UDP) 8

2.3 DVR - Distance Vector Routing (DVR_TCP / DVR_UDP) 9

2.4 Stop-and-Wait ARQ (StopAndWait_TCP / StopAndWait_UDP) 10

2.5 Go-Back-N ARQ (GoBackN_TCP / GoBackN_UDP) 11

2.6 Selective Repeat ARQ (SelectiveRepeat_TCP / SelectiveRepeat_UDP) 13

2.7 FTP - File Transfer (FTP_TCP / FTP_UDP) 14

2.8 SMTP - Mail Sending (SMTP_TCP / SMTP_UDP) 15

2.9 Leaky Bucket - Traffic Shaping (LeakyBucket_TCP / LeakyBucket_UDP) 16

PART 3 - QUICK REFERENCE 19

3.1 Port table 19

3.2 ARQ algorithms compared 19

3.3 How to compile and run 19

PART 1 - TCP AND UDP FOUNDATIONS

Every program in this repository follows one of two skeletons: a TCP (stream) skeleton or a UDP (datagram) skeleton. Once you understand these two structures, you already understand 70% of every file - the remaining 30% is the experiment-specific logic inside the loop. All programs use localhost, plain-text messages, single-class main() files, and only java.io.* and java.net.*.

1.1 TCP vs UDP at a glance

Feature	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Guaranteed delivery, order, retransmission	No guarantees - packets may be lost, reordered or duplicated
Data unit	Byte stream (programs read it as lines)	Datagram (byte-array packet)
Java classes	ServerSocket, Socket, InputStream / OutputStream	DatagramSocket, DatagramPacket
Addressing	Server learns client via accept()	Every datagram carries the sender's address; replies must reuse it
Lab ports	6x01 (DVR 6001 ... Calc 6801)	6x02 (DVR 6002 ... Calc 6802)

1.2 Common TCP SERVER structure

Every *_TCP_Server file (Calculator, Chat, DVR, SW, GBN, SR, FTP, SMTP, LeakyBucket) is built from this exact sequence:

```
import java.io.*;
import java.net.*;

ServerSocket serverSocket = new ServerSocket(PORT);    // 1. bind port, listen
Socket socket = serverSocket.accept();                 // 2. BLOCK until one client connects
BufferedReader in = new BufferedReader(                // 3. text reader on the
        new InputStreamReader(socket.getInputStream())); //    socket's input stream
PrintWriter out = new PrintWriter(                     // 4. text writer (autoflush) on
        socket.getOutputStream(), true);               //    the socket's output stream

// 5. protocol loop: read a line, process it, reply with a line
String line = in.readLine();
if (line == null || line.equals("QUIT")) ...           //    null => client disconnected
out.println(reply);

// 6. teardown
socket.close();
serverSocket.close();
```

- ServerSocket(port) opens the port and puts the server in listening state.
- accept() blocks until a client connects and then returns a dedicated Socket for that ONE client.

These labs serve one client at a time - there are no threads; after QUIT/disconnect the program exits.

- The socket itself carries only bytes. InputStreamReader + BufferedReader converts them into text lines (readLine()); PrintWriter(..., true) uses autoflush so every println() is sent immediately.

- readLine() returning null is the TCP idiom for 'the other side closed the connection' - every server checks it.
- Sessions end with text sentinels: QUIT, END or exit depending on the program.

1.3 Common TCP CLIENT structure

```
Socket socket = new Socket("localhost", PORT);         // connect (3-way handshake)
BufferedReader in = new BufferedReader(
        new InputStreamReader(socket.getInputStream()));
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
Scanner scanner = new Scanner(System.in);              // keyboard input

out.println(message);                                  // send a line
String reply = in.readLine();                          // wait for the reply line

socket.close();
```

- new Socket(host, port) performs the TCP connection - it blocks until the server accepts.
- The same BufferedReader/PrintWriter pair is used for the whole session.
- Scanner(System.in) reads user input; the loop is usually: read input -> send -> print reply.

1.4 Common UDP SERVER structure

```
DatagramSocket socket = new DatagramSocket(PORT);      // bind and wait (NO accept)
byte[] buffer = new byte[1024];

DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
socket.receive(packet);                                // BLOCK for one datagram
String msg = new String(packet.getData(), 0, packet.getLength());  // decode valid bytes

byte[] data = reply.getBytes();
socket.send(new DatagramPacket(data, data.length,
        packet.getAddress(), packet.getPort()));       // reply to the sender
```

- No connection exists - the server simply waits for datagrams from anyone.
- The server learns who the client is FROM each incoming packet (getAddress()/getPort()) and must put that same address into every reply.
- Decode only packet.getLength() bytes: the buffer is reused and may contain stale trailing bytes.
- The same DatagramSocket is used for every send and receive.

1.5 Common UDP CLIENT structure

```
DatagramSocket socket = new DatagramSocket();          // random local port
InetAddress address = InetAddress.getByName("localhost");

byte[] data = message.getBytes();
socket.send(new DatagramPacket(data, data.length, address, PORT));  // one datagram

DatagramPacket reply = new DatagramPacket(new byte[1024], 1024);
socket.receive(reply);                                 // wait for the reply datagram
String text = new String(reply.getData(), 0, reply.getLength());
```

- The client gets a random local port automatically; the server discovers it from the datagram.
- Each send() is one independent datagram - UDP itself gives no guarantee of delivery or order.

That is why the ARQ programs explicitly simulate loss and retransmission.

1.6 Common mechanisms used across the programs

a) Simulated packet/ACK loss (ARQ programs)

```
Random random = new Random();
double loss = 0.30;                         // 30% loss probability
if (random.nextDouble() < loss) {
    System.out.println(frame + " lost");    // simulated loss: do NOT send
} else {
    out.println(frame);                     // actually send
}
```

The sender drops ~30% of frames (frameLoss) and the receiver drops ~30% of ACKs (ackLoss). A 'lost' packet is simply never sent - the program only prints that it was lost.

b) Timer and retransmission (ARQ programs)

```
socket.setSoTimeout(2000);                  // 2-second timer
try {
    ack = in.readLine();                    // TCP  (UDP: socket.receive(packet))
} catch (SocketTimeoutException e) {
    // timer expired -> retransmit
}
```

setSoTimeout(2000) turns every read into a 2-second timer. If nothing arrives, a SocketTimeoutException is thrown and the catch block performs the retransmission. This is the 'timer' of the Stop-and-Wait / Go-Back-N / Selective Repeat protocols.

c) Text sentinels

- QUIT / exit - ends a session (calculator, chat, ARQ, FTP, SMTP, leaky bucket).
- END - terminates multi-line payloads: FTP file contents, DVR routing tables.
- TCP also detects end-of-session with readLine() == null (client closed the socket).

PART 2 - THE 9 PROGRAMS AND THEIR ALGORITHMS

Each section below contains: (1) what the program does, (2) how the server and client work, (3) the TCP vs UDP differences, and (4) complete step-by-step algorithms in the format 'Step 1: Start ... Step n: Stop'.

2.0 Repository map - files and ports

#	Program	TCP files (port)	UDP files (port)
1	DVR - Distance Vector Routing	DVR_TCP_Server/Client (6001)	DVR_UDP_Server/Client (6002)
2	Stop-and-Wait ARQ	SW_TCP_Server/Client (6101)	SW_UDP_Server/Client (6102)
3	Go-Back-N ARQ	GBN_TCP_Server/Client (6201)	GBN_UDP_Server/Client (6202)
4	Selective Repeat ARQ	SR_TCP_Server/Client (6301)	SR_UDP_Server/Client (6302)
5	FTP	FTP_TCP_Server/Client (6401)	FTP_UDP_Server/Client (6402)
6	SMTP	SMTP_TCP_Server/Client (6501)	SMTP_UDP_Server/Client (6502)
7	Leaky Bucket	LeakyBucket_TCP_Server/Client (6601)	LeakyBucket_UDP_Server/Client (6602)
8	Chatting	Chat_TCP_Server/Client (6701)	Chat_UDP_Server/Client (6702)
9	Calculator	Calculator_TCP_Server/Client (6801)	Calculator_UDP_Server/Client (6802)

FTP also ships a server/ folder (with sample.txt) in both FTP_TCP and FTP_UDP for LIST/DOWNLOAD testing.

2.1 Calculator (Calculator_TCP / Calculator_UDP)

What it does

The simplest program and the best template of the TCP/UDP pattern. The client sends arithmetic expressions such as '5 + 3'; the server parses them, computes the result and sends it back. The session repeats until the client types 'exit'.

How it works

- Client: reads an expression from the keyboard and sends it; prints the result returned by the server.
- Server: calculate() splits the expression on spaces into exactly 3 tokens: number, operator, number. It supports +, -, *, / and returns error strings for malformed input ('Use: number operator number'), division by zero ('Cannot divide by zero') and unknown operators ('Invalid operator').

TCP vs UDP differences

Logic is identical in both versions - only the socket boilerplate differs. TCP streams lines; UDP wraps each expression and each result in one datagram.

Algorithm: Calculator - SERVER

Step 1: Start.

Step 2: Initialize socket - TCP: create a ServerSocket on port 6801 and call accept() to wait for a client; UDP: create a DatagramSocket bound to port 6802.

Step 3: Enter the service loop and wait for an expression - TCP: read a line with readLine(); UDP: receive a DatagramPacket and convert its bytes to a String using packet.getLength().

Step 4: If the expression is 'exit' (case-insensitive), go to Step 7.

Step 5: Split the expression on spaces. If it does not have exactly 3 tokens (number, operator, number), reply 'Use: number operator number' and go back to Step 3.

Step 6: Parse token 1 and token 3 as double and apply the operator: +, -, *, / (for / with second operand 0 reply 'Cannot divide by zero'; for any other operator reply 'Invalid operator'). Send the result back to the client - TCP: println on the socket; UDP: datagram to packet.getAddress()/getPort(). Go to Step 3.

Step 7: Close the socket(s).

Step 8: Stop.

Algorithm: Calculator - CLIENT

Step 1: Start.

Step 2: Initialize socket - TCP: Socket("localhost", 6801); UDP: DatagramSocket() with address 'localhost'.

Step 3: Print the prompt 'Enter expression (example: 5 + 3)' and read the expression from the keyboard.

Step 4: Send the expression to the server.

Step 5: If the expression is 'exit', go to Step 7.

Step 6: Receive the server's reply and print 'Result: <answer>'. Go back to Step 3.

Step 7: Close the socket.

Step 8: Stop.

2.2 Chat (Chat_TCP / Chat_UDP)

What it does

A two-way, strictly turn-based chat between the server operator and the client user. One side sends a message, the other reads it, types a reply and sends it back. Either side can end the conversation with 'exit'.

How it works

- Server: (TCP only) sends the greeting 'Connected to chat server'. Then loops: receive message -> print 'Client: <msg>' -> read operator reply from keyboard -> send reply.
- Client: loops: read user message -> send it -> receive reply -> print 'Server: <reply>'.
- Neither program reads the keyboard and the socket at the same time, so messages alternate.

TCP vs UDP differences

TCP exchanges lines over the persistent connection; UDP sends each message as a datagram and replies to the address found in the incoming packet.

Algorithm: Chat - SERVER

Step 1: Start.

Step 2: Initialize socket - TCP: ServerSocket on port 6701 and accept() the client; UDP: DatagramSocket on port 6702.

Step 3: (TCP only) Send the greeting 'Connected to chat server'.

Step 4: Receive a message from the client and print 'Client: <message>'.

Step 5: If the message is 'exit' (case-insensitive), go to Step 8.

Step 6: Read the server operator's reply from the keyboard.

Step 7: Send the reply. If the reply is 'exit', go to Step 8; otherwise go back to Step 4.

Step 8: Close the socket(s).

Step 9: Stop.

Algorithm: Chat - CLIENT

Step 1: Start.

Step 2: Initialize socket - TCP: Socket("localhost", 6701); UDP: DatagramSocket() with address 'localhost'. (TCP) Print the server greeting.

Step 3: Read a message from the keyboard and send it to the server.

Step 4: If the message is 'exit', go to Step 7.

Step 5: Receive the server's reply and print 'Server: <reply>'.

Step 6: If the reply is 'exit', go to Step 7; otherwise go back to Step 3.

Step 7: Close the socket.

Step 8: Stop.

2.3 DVR - Distance Vector Routing (DVR_TCP / DVR_UDP)

What it does

Simulates one update step of the Distance Vector routing algorithm. The server plays Router A with a known distance vector; the client supplies the distance vector of neighbor Router B. Router A applies the Bellman-Ford relaxation D(A, i) = min(D(A, i), c(A,B) + D(B, i)) and returns its improved routing table.

How it works

- Router A's initial table is fixed in the code: A = 0, B = 2, C = 7; the link cost A-B is 2.
- The client enters B's vector, e.g. '2 0 1' (B reaches A in 2, itself in 0, C in 1).
- The server computes for every destination: throughB = 2 + neighbor[i]; if throughB is cheaper than the current distance, the table is updated. With the example the table becomes A 0, B 2, C 3.
- TCP returns the table as several lines terminated by 'END'; UDP returns one multi-line datagram.

TCP vs UDP differences

TCP sends an instruction line first and streams the table line-by-line until END; UDP has no handshake - one datagram in, one datagram out.

Algorithm: DVR - Distance Vector Routing - SERVER

Step 1: Start.

Step 2: Initialize socket - TCP: ServerSocket on port 6001 and accept(); UDP: DatagramSocket on port 6002.

Step 3: Initialize Router A's distance vector: distance = {0, 2, 7} for nodes {A, B, C}; set link cost linkCost = 2 (cost of link A-B).

Step 4: (TCP only) Send the format instruction line to the client.

Step 5: Receive the neighbor's (Router B's) distance vector as space-separated integers and parse it into neighbor[].

Step 6: For each destination i: compute throughB = linkCost + neighbor[i]; if throughB < distance[i], set distance[i] = throughB (Bellman-Ford relaxation step).

Step 7: Send the updated routing table - TCP: one line per node ('A 0', 'B 2', 'C 3') followed by 'END'; UDP: a single datagram containing all lines.

Step 8: Close the socket(s).

Step 9: Stop.

Algorithm: DVR - Distance Vector Routing - CLIENT

Step 1: Start.

Step 2: Initialize socket - TCP: Socket("localhost", 6001); UDP: DatagramSocket() with address 'localhost'.

Step 3: (TCP only) Read and print the server's format instruction.

Step 4: Read the neighbor routing vector from the keyboard (example: 2 0 1) and send it to the server.

Step 5: Receive and print the updated routing table - TCP: print lines until 'END'; UDP: print the received datagram.

Step 6: Close the socket.

Step 7: Stop.

2.4 Stop-and-Wait ARQ (StopAndWait_TCP / StopAndWait_UDP)

What it does

The simplest reliable-transfer protocol. The sender sends exactly ONE frame and then waits for its ACK before sending the next frame. Sequence numbers alternate 0/1, a 2-second timer detects lost frames or lost ACKs, and both sides simulate 30% loss.

How it works

- Sender (client): sends 'FRAME <seq>' (30% simulated loss), starts a 2 s timer and waits for 'ACK <seq+1 mod 2>'. On timeout it retransmits the same frame. Only the correct ACK advances to the next frame.
- Receiver (server): keeps an 'expected' number. A matching frame is accepted and expected flips (0 <-> 1); a duplicate frame is discarded - but the ACK of 'expected' is still (re)sent, which un-blocks a sender whose previous ACK was lost.
- TCP server additionally sends a 'READY' greeting before the frame exchange.

TCP vs UDP differences

Identical protocol logic. TCP carries frames/ACKs as lines on the connection; UDP carries them as datagrams, and the server replies to the address found in each incoming packet.

Algorithm: Stop-and-Wait ARQ - SERVER

Step 1: Start.

Step 2: Initialize socket - TCP: ServerSocket on port 6101 and accept(); UDP: DatagramSocket on port 6102. Create the receive buffer and readers/writers.

Step 3: Set expected = 0 and ackLoss = 0.30. (TCP only) Send 'READY' to the client.

Step 4: Wait for a frame - TCP: readLine(); UDP: receive() a datagram and decode it.

Step 5: If the frame is 'QUIT' (or the stream ended), go to Step 9.

Step 6: Extract the frame number from 'FRAME <n>'.

Step 7: If number == expected: accept the frame and set expected = (expected + 1) % 2; otherwise print 'Duplicate Frame discarded' (the duplicate is dropped, but the ACK below is still sent).

Step 8: Prepare 'ACK <expected>'. With probability 0.30 simulate ACK loss (print 'ACK <n> lost' and do not send); otherwise send the ACK. Go back to Step 4.

Step 9: Close the socket(s).

Step 10: Stop.

Algorithm: Stop-and-Wait ARQ - CLIENT

Step 1: Start.

Step 2: Initialize socket - TCP: Socket("localhost", 6101); UDP: DatagramSocket(). Set frameLoss = 0.30 and timeout = 2000 ms.

Step 3: (TCP only) Read and print the server greeting 'READY'.

Step 4: Read the number of frames n from the keyboard. Set sequence = 0.

Step 5: For each frame i = 0 to n-1, repeat Steps 6 to 9.

Step 6: Send 'FRAME <sequence>' - with probability 0.30 simulate loss (print 'FRAME <seq> lost' and do not send).

Step 7: Start the 2-second timer and wait for an ACK.

Step 8: If an ACK arrives: if ackNo == (sequence + 1) % 2 it is the correct ACK - proceed to Step 9; otherwise ignore it and go back to Step 6. If the timer expires (SocketTimeoutException), print 'Timeout! Resending' and go back to Step 6.

Step 9: Set sequence = (sequence + 1) % 2. If more frames remain, go to Step 6 for the next frame.

Step 10: Send 'QUIT'.

Step 11: Close the socket.

Step 12: Stop.

2.5 Go-Back-N ARQ (GoBackN_TCP / GoBackN_UDP)

What it does

Pipelined reliable transfer. The sender may have up to N unacknowledged frames in flight (the window). ACKs are cumulative; the receiver has a window of 1 and discards any out-of-order frame. On timeout the sender GOES BACK and retransmits ALL outstanding frames from the oldest

unacknowledged one.

How it works

- Sender (client): user enters n frames and window size w. Pointers: base = oldest unacked frame, next = next frame to send. It sends all frames up to base + w without waiting, then listens for ACKs. 'ACK k' means everything below k is received, so base jumps to k (window slides).
- Timeout: retransmit every outstanding frame frames[base] .. frames[next-1].
- Receiver (server): accepts only number == expected (then sends cumulative 'ACK expected'); duplicates (number < expected) are discarded but re-acknowledged; out-of-order frames (number > expected) are discarded silently - no buffering.
- Both sides simulate 30% loss (frames on the client, ACKs on the server).

TCP vs UDP differences

Same algorithm. TCP sends frames/ACKs as lines; UDP wraps each in a datagram and factors the send logic into helper functions sendFrame()/sendAck().

Algorithm: Go-Back-N ARQ - SERVER

Step 1: Start.

Step 2: Initialize socket - TCP: ServerSocket on port 6201 and accept(); UDP: DatagramSocket on port 6202.

Step 3: Set expected = 0 and ackLoss = 0.30.

Step 4: Wait for a frame. If it is 'QUIT' (or the stream ended), go to Step 9.

Step 5: Extract the frame number.

Step 6: If number == expected: accept the frame, set expected = expected + 1, and send the cumulative 'ACK <expected>' (with 0.30 probability of simulated ACK loss).

Step 7: If number < expected: it is a duplicate - discard it, but resend 'ACK <expected>' (again with 0.30 loss chance) so the sender can advance.

Step 8: If number > expected: out-of-order frame - discard it and send nothing (the receiver does not buffer). Go back to Step 4.

Step 9: Close the socket(s).

Step 10: Stop.

Algorithm: Go-Back-N ARQ - CLIENT

Step 1: Start.

Step 2: Initialize socket - TCP: Socket("localhost", 6201); UDP: DatagramSocket(). Set frameLoss = 0.30 and timeout = 2000 ms.

Step 3: Read the number of frames n and the window size w. Build frames FRAME 0 .. FRAME n-1. Set base = 0, next = 0.

Step 4: While base < n, repeat Steps 5 to 8.

Step 5: Send every frame inside the window: while next < n and next < base + w, send frames[next] (with 0.30 simulated loss) and set next = next + 1.

Step 6: Start the 2-second timer and wait for an ACK.

Step 7: If 'ACK k' arrives and base < k <= next: set base = k (cumulative ACK slides the window forward). Go to Step 4.

Step 8: On timeout (SocketTimeoutException): GO BACK N - retransmit all outstanding frames frames[base] .. frames[next-1], each with 0.30 simulated loss. Go to Step 4.

Step 9: Print 'All frames sent successfully' and send 'QUIT'.

Step 10: Close the socket.

Step 11: Stop.

2.6 Selective Repeat ARQ (SelectiveRepeat_TCP / SelectiveRepeat_UDP)

What it does

The most advanced ARQ. The receiver BUFFERS valid out-of-order frames inside its window (size 4) instead of discarding them, signals gaps with NAKs, and the sender retransmits ONLY the missing frame - not the whole window. ACKs remain cumulative.

How it works

- Sender (client): keeps acked[] per frame plus base/next window pointers. On 'ACK k' it marks frames base .. k-1 as acked and slides base. On 'NAK k' it retransmits only frame k. On timeout it retransmits only the oldest unacked frame frames[base].
- Receiver (server): a frame inside [expected, expected + window) is buffered (received[n] = true). If a higher frame arrives while 'expected' is missing, it sends 'NAK expected'. Whenever possible it delivers buffered frames in order, advances expected and sends one cumulative 'ACK expected'.
- Duplicates and old frames are answered with the current ACK so the sender can recover lost ACKs.

TCP vs UDP differences

The TCP version additionally simulates 30% loss on frames (client) and ACKs (server); the UDP version relies on the same NAK/ACK/timeout logic over datagrams.

Algorithm: Selective Repeat ARQ - SERVER

Step 1: Start.

Step 2: Initialize socket - TCP: ServerSocket on port 6301 and accept(); UDP: DatagramSocket on port 6302.

Step 3: Set expected = 0, receiver window = 4, and received[0..99] = false.

Step 4: Wait for a frame. If it is 'QUIT' (or the stream ended), go to Step 9.

Step 5: Extract the frame number.

Step 6: If expected <= number < expected + window: if the frame is new, buffer it (received[number] = true). If number > expected and received[expected] is false, a gap exists - send 'NAK expected'. Then while received[expected] is true, deliver frames in order and increment expected. If expected advanced, send the cumulative 'ACK expected'. If the frame was a duplicate inside the window, resend 'ACK expected'.

Step 7: If number < expected: old duplicate - discard it and resend 'ACK expected'.

Step 8: If number >= expected + window: the frame lies outside the window - ignore it. Go back to Step 4.

Step 9: Close the socket(s).

Step 10: Stop.

Algorithm: Selective Repeat ARQ - CLIENT

Step 1: Start.

Step 2: Initialize socket - TCP: Socket("localhost", 6301); UDP: DatagramSocket(). Set frameLoss = 0.30 and timeout = 2000 ms.

Step 3: Read the number of frames n and the window size w. Build frames FRAME 0 .. FRAME n-1; set acked[0..n-1] = false, base = 0, next = 0.

Step 4: While base < n, repeat Steps 5 to 9.

Step 5: Send new frames inside the window: while next < n and next < base + w, send frames[next] (with 0.30 simulated loss) and set next = next + 1.

Step 6: Wait up to 2 seconds for a response.

Step 7: If 'ACK k' arrives: mark acked[base .. k-1] = true, then advance base while acked[base] is true. Go to Step 4.

Step 8: If 'NAK k' arrives: retransmit only frame k (if it has not been acked). Go to Step 4.

Step 9: On timeout: retransmit only the oldest unacknowledged frame frames[base]. Go to Step 4.

Step 10: Send 'QUIT'.

Step 11: Close the socket.

Step 12: Stop.

2.7 FTP - File Transfer (FTP_TCP / FTP_UDP)

What it does

A simplified FTP with four text commands: LIST (list server files), UPLOAD, DOWNLOAD and QUIT, using real FTP-style reply codes (220, 250, 404, 221, 500). The server stores files in a local 'server' directory that ships with sample.txt.

How it works

- Server: greets with '220 FTP Server Ready' (TCP), then loops over commands. LIST returns the file names followed by 'END'. UPLOAD <name>: reply READY, read the file content until 'END', write it to server/<name>, reply '250 File uploaded successfully'. DOWNLOAD <name>: reply READY plus the file content plus 'END', or '404 File not found'. QUIT: reply '221 Goodbye'.
- Client: shows a menu (1 LIST, 2 UPLOAD, 3 DOWNLOAD, 4 QUIT). UPLOAD reads the local file and streams it; DOWNLOAD saves the received content locally.

TCP vs UDP differences

TCP transfers the file LINE BY LINE over the persistent connection, using 'END' as the end-of-file marker. UDP cannot stream, so the ENTIRE file travels inside a single datagram (server buffer 4096 bytes, client download buffer 65507 bytes - near the UDP maximum). Real-world FTP is TCP-only; the UDP version is an educational simulation.

Algorithm: FTP - File Transfer - SERVER

Step 1: Start.

Step 2: Initialize socket - TCP: ServerSocket on port 6401 and accept(); UDP: DatagramSocket on port 6402. Create the 'server' directory if missing.

Step 3: (TCP only) Send the greeting '220 FTP Server Ready'.

Step 4: Wait for a command from the client.

Step 5: If the command is 'LIST': send the names of all files in the server folder - TCP: one line per name followed by 'END'; UDP: one datagram containing all names (or 'No files'). Go to Step 4.

Step 6: If the command is 'UPLOAD <name>': reply 'READY'; receive the file content - TCP: lines until 'END'; UDP: a single datagram containing all file bytes; write the content to server/<name>; reply '250 File uploaded successfully'. Go to Step 4.

Step 7: If the command is 'DOWNLOAD <name>': if the file does not exist, reply '404 File not found'; otherwise reply 'READY' and send the file - TCP: line by line followed by 'END'; UDP: the whole file as one datagram. Go to Step 4.

Step 8: If the command is 'QUIT': reply '221 Goodbye' and go to Step 10. For any other command reply '500 Invalid command' and go to Step 4.

Step 9: Close the socket(s).

Step 10: Stop.

Algorithm: FTP - File Transfer - CLIENT

Step 1: Start.

Step 2: Initialize socket - TCP: Socket("localhost", 6401); UDP: DatagramSocket() with address 'localhost'. (TCP) Print the server greeting '220 FTP Server Ready'.

Step 3: Display the menu (1 LIST, 2 UPLOAD, 3 DOWNLOAD, 4 QUIT) and read the choice.

Step 4: Choice LIST: send 'LIST'; receive and print the file names until 'END' (TCP) or from the reply datagram (UDP). Go to Step 3.

Step 5: Choice UPLOAD: read the local file name. If the file does not exist, print 'File not found' and go to Step 3. Otherwise send 'UPLOAD <name>', wait for 'READY', send the file content (TCP: lines then 'END'; UDP: all bytes in one datagram), and print the server confirmation. Go to Step 3.

Step 6: Choice DOWNLOAD: read the file name and send 'DOWNLOAD <name>'. If the reply is '404 File not found', print it and go to Step 3. Otherwise receive the content until 'END' (TCP) or from one datagram (UDP) and save it to a local file. Go to Step 3.

Step 7: Choice QUIT: send 'QUIT' (TCP prints '221 Goodbye').

Step 8: Close the socket.

Step 9: Stop.

2.8 SMTP - Mail Sending (SMTP_TCP / SMTP_UDP)

What it does

Simulates an SMTP mail session with the classic command/response exchange and numeric codes: 220 greeting, HELO -> 250, MAIL FROM -> 250, RCPT TO -> 250, DATA -> 354, body terminated by a lone '.', 250 accepted, QUIT -> 221. Real SMTP is TCP; the UDP version is an educational simulation.

How it works

- Client: reads sender, receiver, subject and message from the keyboard and walks through the SMTP handshake, printing every server reply.
- Server: answers each command with its SMTP code. For DATA, the TCP server reads body lines until a line equal to '.'; the UDP server treats each line (and the final '.') as separate datagrams.

TCP vs UDP differences

TCP keeps one connection for the whole session and reads the message body as lines until '.'. UDP sends every command and every body line as its own datagram; the server answers each with '250 Received' until the '.' datagram arrives, which is answered with '250 Message accepted'.

Algorithm: SMTP - Mail Sending - SERVER

Step 1: Start.

Step 2: Initialize socket - TCP: ServerSocket on port 6501 and accept(); UDP: DatagramSocket on port 6502.

Step 3: (TCP only) Send the greeting '220 SMTP Server Ready'.

Step 4: Wait for a command.

Step 5: If the command starts with 'HELO': reply '250 Hello'. Go to Step 4.

Step 6: If the command starts with 'MAIL FROM': reply '250 Sender OK'. Go to Step 4.

Step 7: If the command starts with 'RCPT TO': reply '250 Receiver OK'. Go to Step 4.

Step 8: If the command is 'DATA': TCP - reply '354 Start mail input; end with .', then read and print mail lines until a line equal to '.' and reply '250 Message accepted'. UDP - reply '354 Ready for message'; each following body-line datagram is answered '250 Received' until the '.' datagram, which is answered '250 Message accepted'. Go to Step 4.

Step 9: If the command is 'QUIT': reply '221 Bye' and go to Step 11. For any other command reply '500 Command not recognized' and go to Step 4.

Step 10: Close the socket(s).

Step 11: Stop.

Algorithm: SMTP - Mail Sending - CLIENT

Step 1: Start.

Step 2: Initialize socket - TCP: Socket("localhost", 6501); UDP: DatagramSocket() with address 'localhost'. (TCP) Read and print the '220 SMTP Server Ready' greeting.

Step 3: Send 'HELO localhost' and print the reply ('250 Hello').

Step 4: Read the sender e-mail address; send 'MAIL FROM:<sender>'; print the reply ('250 Sender OK').

Step 5: Read the receiver e-mail address; send 'RCPT TO:<receiver>'; print the reply ('250 Receiver OK').

Step 6: Send 'DATA' and print the reply (354 ...).

Step 7: Read the subject and the message; send 'Subject: <subject>' followed by the message body.

Step 8: Send the lone '.' line/datagram to end the mail and print the reply ('250 Message accepted').

Step 9: Send 'QUIT' and print the reply ('221 Bye').

Step 10: Close the socket.

Step 11: Stop.

2.9 Leaky Bucket - Traffic Shaping (LeakyBucket_TCP / LeakyBucket_UDP)

What it does

Simulates the leaky bucket traffic-policing algorithm. The client configures a bucket (capacity and leak rate) and then feeds it packet sizes; the server shows which packets are accepted, how much leaks out at a constant rate, and which packets are dropped on overflow.

How it works

- Client: reads bucket capacity, leak rate and the number of packets n; sends the configuration, then sends n packet sizes one by one, printing the server's verdict for each.
- Server: keeps an integer bucket level starting at 0. For each packet: if bucket + size > capacity the packet is DROPPED; otherwise it is added, then leaked = min(leakRate, bucket) units leak out immediately and the new level is reported. 'END' finishes the session with 'DONE'.

TCP vs UDP differences

TCP sends the configuration as one line and each verdict as two reply lines (accepted/leaked); UDP exchanges one datagram per message and combines the verdict into a single reply, sending 'READY' after receiving the configuration.

Algorithm: Leaky Bucket - Traffic Shaping - SERVER

Step 1: Start.

Step 2: Initialize socket - TCP: ServerSocket on port 6601 and accept(); UDP: DatagramSocket on port 6602.

Step 3: Receive the configuration 'capacity leakRate' and parse both integers. Set bucket = 0. (UDP only) Reply 'READY'.

Step 4: Wait for the next message (a packet size or 'END').

Step 5: If the message is 'END': reply 'DONE' and go to Step 9.

Step 6: If bucket + size > capacity: reply 'Packet <size> DROPPED (bucket overflow)'. Go to Step 4.

Step 7: Otherwise accept the packet: bucket = bucket + size, and reply that the packet was accepted with the new bucket level.

Step 8: Leak: leaked = min(leakRate, bucket); bucket = bucket - leaked; reply with the leaked amount and the remaining bucket level (TCP sends this as a separate line; UDP combines it into one reply). Go to Step 4.

Step 9: Close the socket(s).

Step 10: Stop.

Algorithm: Leaky Bucket - Traffic Shaping - CLIENT

Step 1: Start.

Step 2: Initialize socket - TCP: Socket("localhost", 6601); UDP: DatagramSocket() with address 'localhost'.

Step 3: Read the bucket capacity, the leak rate and the number of packets n from the keyboard.

Step 4: Send the configuration '<capacity> <leakRate>'. (TCP) Print the first server line; (UDP) print the 'READY' reply.

Step 5: For each of the n packets: read the packet size, send it, and print the server's verdict line(s).

Step 6: Send 'END' and print the final 'DONE' reply.

Step 7: Close the socket.

Step 8: Stop.

PART 3 - QUICK REFERENCE

3.1 Port table

Program	TCP port	UDP port
DVR (Distance Vector Routing)	6001	6002
Stop-and-Wait ARQ	6101	6102
Go-Back-N ARQ	6201	6202
Selective Repeat ARQ	6301	6302
FTP	6401	6402
SMTP	6501	6502
Leaky Bucket	6601	6602
Chatting	6701	6702
Calculator	6801	6802

3.2 ARQ algorithms compared

Aspect	Stop-and-Wait	Go-Back-N	Selective Repeat
Sender window	1	N	N
Receiver window	1	1	4 (buffers)
Sequence numbers	alternate 0 / 1	0 .. n-1	0 .. n-1
ACK type	individual (next expected)	cumulative	cumulative + NAK
Out-of-order frame	impossible (one at a time)	discarded	buffered
On loss / timeout	retransmit that frame	retransmit ALL from base	retransmit only NAK'd / oldest unacked

3.3 How to compile and run

```
cd StopAndWait_TCP
javac *.java                  (or: javac SW_TCP_Server.java SW_TCP_Client.java)

Terminal 1:  java SW_TCP_Server     <- always start the server first
Terminal 2:  java SW_TCP_Client     <- then the client
```

- All programs use localhost, so server and client run on the same computer in two separate terminals.
- Always run the Server/Receiver first, then the Client/Sender.
- UDP versions intentionally simulate packet/ACK loss (30%) where appropriate.
- FTP and SMTP are TCP protocols in the real world; their UDP versions are educational simulations of FTP/SMTP-style command exchanges over UDP.
- FTP demos can use the included server/sample.txt for LIST and DOWNLOAD.
- ARQ notes (Module 2): Stop-and-Wait uses sequence numbers, timers and retransmission; Go-Back-N uses a sender window, cumulative ACKs, receiver window 1 and retransmission from the oldest outstanding frame; Selective Repeat buffers valid out-of-order frames and retransmits only the missing/oldest unacknowledged frame.